

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «Логика и ОА в ИЗ»
на тему «Динамические списки»

Выполнили:
студенты группы 25BBB1
Димов Н. А.
Бардычев А.А.

Принял:
к.т.н., доцент Юрова О.В.
ст. преподаватель Деев М. В.

Пенза 2026

Цель работы

Изучение динамических структур данных языка Си на примере односвязных списков, освоение работы с указателями и динамической памятью (`malloc`, `free`), а также реализация на основе односвязного списка трёх структур данных: приоритетной очереди, очереди (FIFO) и стека (LIFO).

Лабораторное задание

На основе приведённого в методических указаниях кода односвязного списка выполнить следующие задания:

Задание 1: Реализовать приоритетную очередь путём добавления элемента в список в соответствии с приоритетом объекта (объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Задание 2: Реализовать структуру данных «Очередь» (принцип FIFO – первым пришёл, первым вышел).

Задание 3: Реализовать структуру данных «Стек» (принцип LIFO – последним пришёл, первым вышел).

Описание метода решения задач

Во всех трёх заданиях используется одна и та же базовая структура — односвязный линейный динамический список. Элемент списка описывается структурой, в которую по сравнению с исходным вариантом из методички добавлено целочисленное поле `priority` (имеет смысл только для приоритетной очереди). Доступ к списку осуществляется через глобальные указатели `head` (на первый элемент) и `last` (на последний). Каждое задание перед началом работы обнуляет оба указателя, поэтому структуры данных не пересекаются между собой. Память под новый элемент выделяется функцией `malloc` в `get_struct` с обязательной проверкой результата на `NULL`, а освобождается функцией `free` при удалении элемента или при завершении работы через `free_all`. Из исходного кода методички без изменений перенесены функции `review` (просмотр списка), `find` (поиск по содержимому) и `del` (удаление по содержимому); дополнительно введены `free_all` (полное освобождение списка) и `review_rq` (просмотр приоритетной очереди с выводом приоритета).

Задание 1 (приоритетная очередь, функция `pq_add`).

Список всё время поддерживается упорядоченным по убыванию приоритета. После ввода имени и приоритета возможны два сценария. Если список пуст либо приоритет нового элемента строго больше приоритета головы — новый элемент становится головой списка. В противном случае выполняется проход по списку двумя указателями (`prev` — предыдущий, `cur` — текущий) до тех пор, пока приоритет текущего элемента больше или равен приоритету нового; после этого элемент вставляется в разрыв между `prev` и `cur`. Если `cur == NULL`, вставка происходит в конец, и указатель `last` обновляется. Использование нестрогого условия «больше или равен» обеспечивает устойчивость: объекты с одинаковым приоритетом сохраняют порядок поступления. В результате первым в списке всегда оказывается объект с наибольшим приоритетом, и именно он извлекается функцией `extract`.

Задание 2 (очередь, функции `queue_add` и `queue_remove`).

Добавление нового элемента выполняется в конец списка через указатель `last` за время $O(1)$: если очередь пуста, `head` и `last` начинают указывать на новый узел; иначе выполняется `last->next = p` и `last = p`. Извлечение производится из начала: указатель `head` переносится на следующий элемент, а бывший первый узел освобождается. Если очередь после извлечения стала пустой, указатель `last` обнуляется. Такой порядок гарантирует, что первым из очереди уходит элемент, добавленный раньше всех.

Задание 3 (стек, функции `stack_push` и `stack_pop`).

Операция `push` добавляет элемент в начало списка: сначала выполняется `p->next = head`, затем `head = p`. Операция `pop` снимает элемент с вершины, то есть из головы списка, и освобождает память. Обе операции выполняются за $O(1)$; первым из стека извлекается элемент, добавленный последним.

Сводка операций: В приоритетной очереди элемент вставляется в позицию, определяемую его приоритетом (сложность добавления $O(n)$), а извлекается из головы. В очереди (FIFO) добавление идёт в конец за $O(1)$, извлечение — из головы. В стеке (LIFO) и добавление, и извлечение выполняются из головы за $O(1)$.

Пояснительный текст к программе

Структура программы.

При запуске выводится меню выбора задания (1 — приоритетная очередь, 2 — очередь, 3 — стек), после чего управление передаётся соответствующей функции `task1`, `task2` или `task3`. Каждая из них содержит собственный цикл с меню операций; при выборе пункта «Exit» список полностью освобождается функцией `free_list` и работа завершается.

Основные функции.

`get_struct` создаёт элемент списка: выделяет память с проверкой результата и запрашивает у пользователя имя (и приоритет — для приоритетной очереди). Функции `review` и `review_rq` выводят список от головы к концу, причём вторая дополнительно печатает приоритет каждого элемента. `find` ищет узел по содержимому и возвращает указатель на него. `del` удаляет все элементы с заданным содержимым, корректно перестраивая указатели `head`, `last` и связи между узлами. `free_list` освобождает память всех элементов списка. Функция `rq_add` вставляет элемент в приоритетную очередь в позицию, соответствующую его приоритету. Функции `queue_add` и `queue_remove` реализуют добавление в конец и извлечение из начала очереди. Функции `stack_push` и `stack_pop` добавляют элемент на вершину и снимают его с вершины стека. Функции `task1`, `task2`, `task3` содержат меню и основной цикл работы с соответствующей структурой данных.

Динамическое управление памятью.

Все элементы создаются функцией `malloc` с обязательной проверкой результата на `NULL` и освобождаются функцией `free` при удалении элемента, извлечении из очереди/стека или при выходе из режима работы. Это исключает утечки памяти. Обработка граничных случаев.

При работе с пустым списком функции просмотра, поиска и удаления выводят сообщение «List is empty»; при извлечении из пустой очереди или стека — «Queue is empty» либо «Stack is empty»; при отсутствии искомого элемента — «Not found». При удалении последнего элемента, а также при извлечении единственного элемента указатель `last` корректируется. Полный текст программы приведён в приложении.

Результаты работы программы

Приоритетная очередь.

Последовательно добавлены объекты TaskA (приоритет 2), TaskB (5), TaskC (2) и TaskD (5). При просмотре объекты располагаются по убыванию приоритета, а объекты с одинаковым приоритетом — в порядке добавления. Затем выполнены поиск объекта TaskC и удаление объекта TaskA; после удаления очередь остаётся упорядоченной.

Очередь.

Добавлены объекты Alpha, Beta и Gamma; извлечение происходит в порядке добавления — первым уходит Alpha. Дополнительно проверены поиск существующего элемента и удаление отсутствующего элемента (Beta уже был извлечён ранее).

Стек.

Добавлены объекты One, Two и Three; извлечение происходит в обратном порядке — первым снимается Three. Попытка извлечения из пустого стека приводит к сообщению об ошибке.

Выводы

В ходе выполнения лабораторной работы №3 были изучены и закреплены:

1. Принципы организации односвязных динамических списков и работа с их элементами через указатели.
2. Функции динамического выделения и освобождения памяти (malloc, free) при создании и удалении элементов списка.
3. Алгоритм вставки элемента в упорядоченный список, позволяющий реализовать приоритетную очередь с сохранением порядка поступления элементов с равным приоритетом.
4. Реализация структур данных «Очередь» (FIFO) и «Стек» (LIFO) на основе односвязного списка: очередь требует указателя на конец списка для добавления за $O(1)$, стек работает только с головой списка.
5. Обработка граничных случаев (пустой список, удаление единственного или последнего элемента) и корректное освобождение памяти при завершении работы.